Physics Simulation for Computer Graphics

03 Rigid Body Dynamics

Rigid Bodies

Points vs. Rigid Bodies

- For particles:

  Position $\mathbf{x}_\mathrm{p}$

  Velocity $\mathbf{v}_\mathrm{p}$

  Mass M

- Dynamics:

$$\frac{d\mathbf{x}_p(t)}{dt} = \mathbf{v}(\mathbf{x}_p, t)$$

$$\frac{d\mathbf{v}_p(t)}{dt} = \frac{\mathbf{f}(\mathbf{x}_p, t)}{m_p}$$

- For a rigid body:
  (Translational Motion+ Rotational Motion)
  - Position $\mathbf{x}_\mathrm{c}$
  - Status of Rotation→ Orientation
  - Velocity $\mathbf{v}_\mathrm{c}$
  - How Rotation Changes → Angular velocity
  - Mass M
  - How easily does the Rigid body rotates?
    → Inertia tensor

Implementation

$\mathbf{v_c}$

$\mathbf{x_c}$

$\mathbf{f}_i \longleftarrow \text{Force}(\mathbf{x}_i, \mathbf{v}_i)$

$\mathbf{f} \longleftarrow \sum \mathbf{f}_i$

$\mathbf{v_c} \longleftarrow \mathbf{v_c} + \Delta t M^{-1}\mathbf{f} \longleftarrow$ Explicit

$\mathbf{x_c} \longleftarrow \mathbf{x_c} + \Delta t \mathbf{v} \longleftarrow$ Implicit

Translational Motion

$\mathbf{v_c}$

$\mathbf{x_c}$

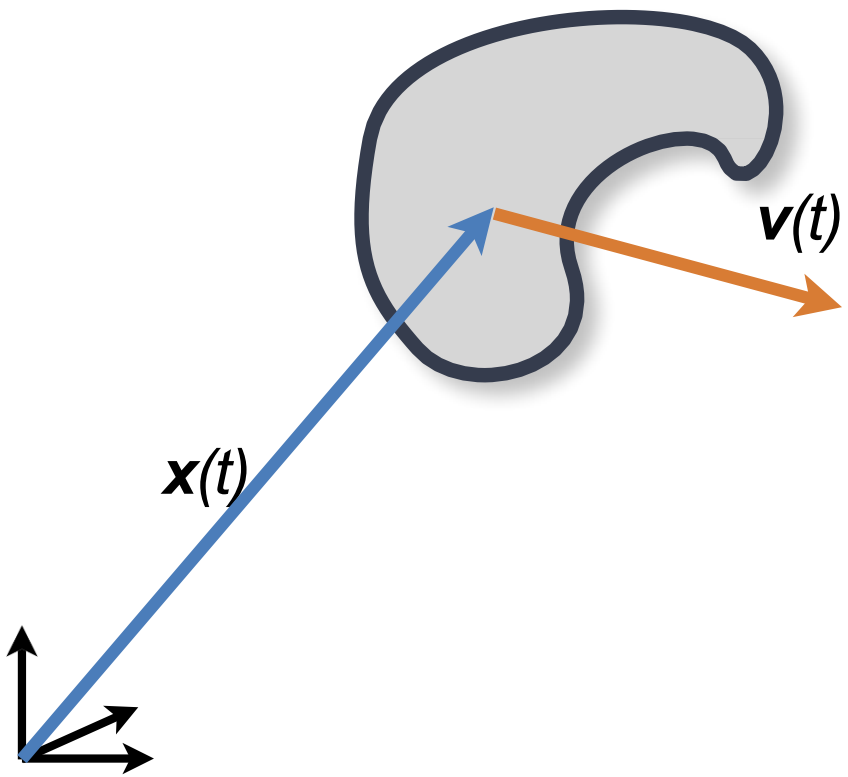
?

?

Rotational Motion

?

Translational Motion (motion of the mass center)



- Center of mass behaves like a mass point with total mass of the body

$$\mathbf{x_c} = \frac{\sum m_i \mathbf{x}_i}{\sum m_i}$$

$$\mathbf{v_c} = \frac{\sum m_i \mathbf{v}_i}{\sum m_i}$$

$$\mathbf{a_c} = \frac{\sum m_i \mathbf{a}_i}{\sum m_i}$$

$$\mathbf{f}_i \leftarrow \mathrm{Force}(\mathbf{x}_i, \mathbf{v}_i)$$

$$\mathbf{f} \leftarrow \sum \mathbf{f}_i$$

$$\mathbf{v_c} \leftarrow \mathbf{v_c} + \Delta t M^{-1} \mathbf{f}$$

$$\mathbf{x_c} \leftarrow \mathbf{x_c} + \Delta t \mathbf{v_c}$$

Implementation

$\mathbf{v_c}$

$\mathbf{x_c}$

$\mathbf{f}_i \leftarrow \text{Force}(\mathbf{x}_i, \mathbf{v}_i)$

$\mathbf{f} \leftarrow \sum \mathbf{f}_i$

$\mathbf{v_c} \leftarrow \mathbf{v_c} + \Delta t M^{-1}\mathbf{f}$ ⟵ Explicit

$\mathbf{x_c} \leftarrow \mathbf{x_c} + \Delta t \mathbf{v_c}$ ⟵ Implicit

Translational Motion

$\mathbf{v_c}$

$\mathbf{x_c}$

?

Calculate Inertia tensor, torque

Update angular velocity

Updater orientation

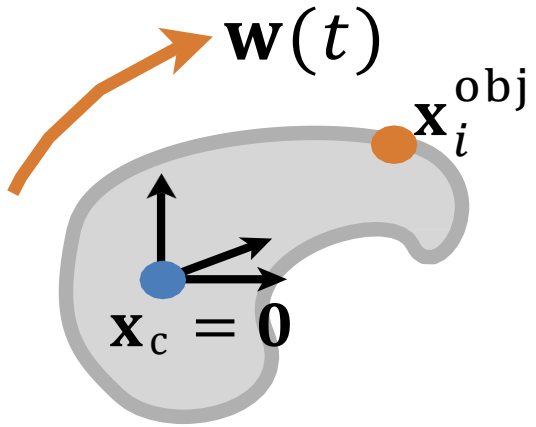
Rotational Motion

?

Local Space to World Space

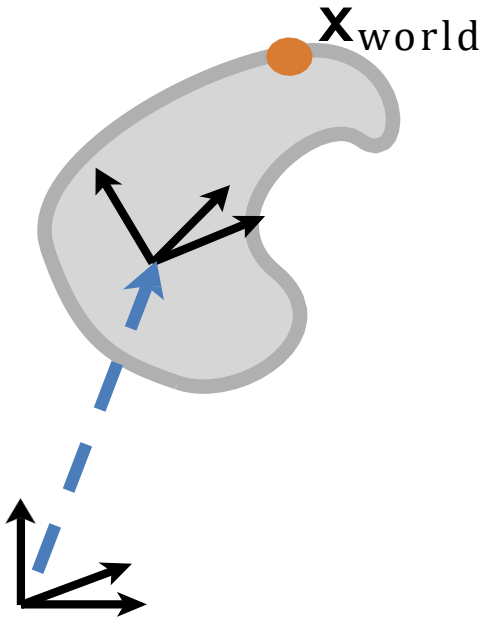
- Orientation: rotation around center of mass, for e.g., a rotation Matrix R
- Angular velocity ($\mathbf{w}$)

Local space



(For convenience, keep center of mass at zero.)

World space



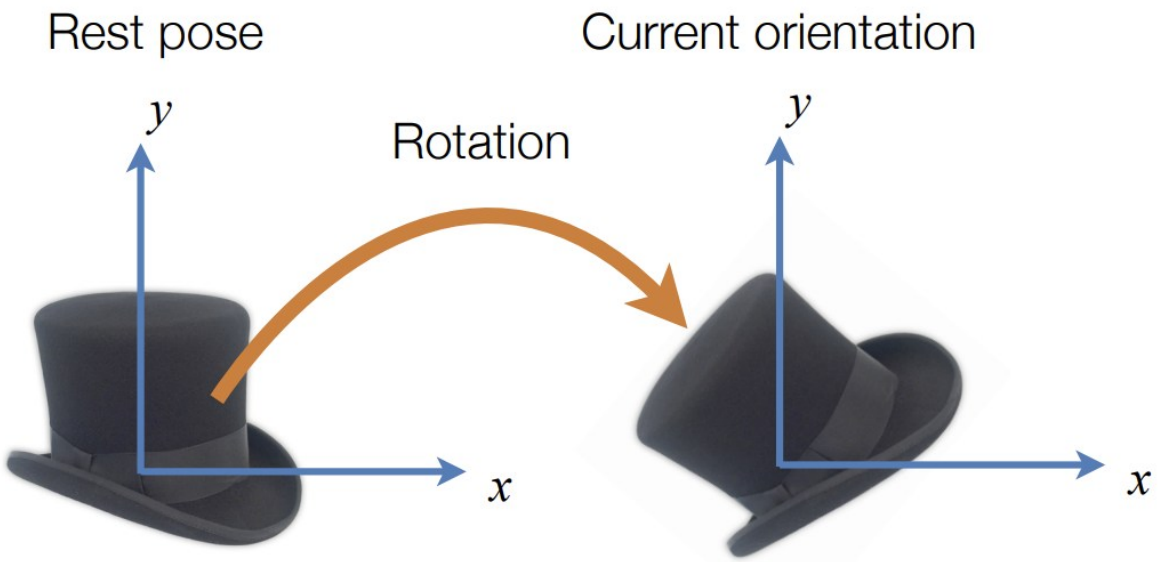
$$\mathbf{x}_i^{\text{world}} = \mathbf{x}_c^{\text{world}}(t) + R(t)\,\mathbf{x}_i^{\text{obj}}$$

$$\mathbf{v}_i^{\text{world}} = \mathbf{v}_c^{\text{world}}(t) + \mathbf{w}(t) \times \mathbf{x}_i^{\text{obj}}$$

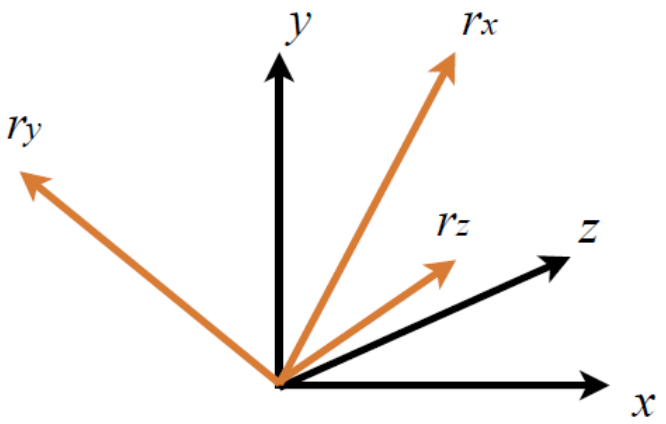
Rotational Motion:
1. Orientation

Orientation Representations

- Rotation Matrix
- Axis and Angles:
  - Fixed Angles
  - Euler Angles
- Quaternions



Rotation Matrix

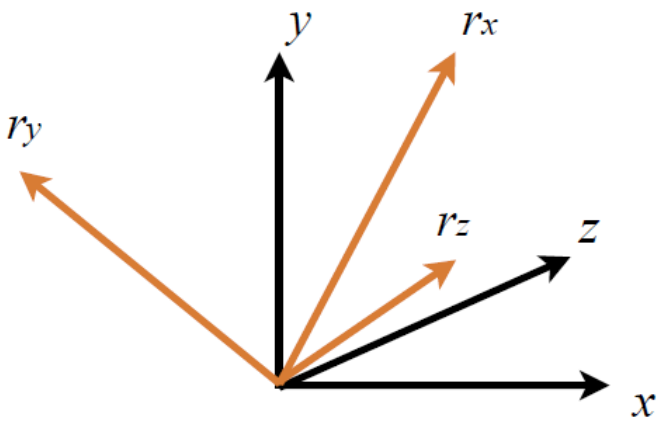


$$\mathbf{R} = \begin{bmatrix} r_{x0} & r_{y0} & r_{z0} \\ r_{x1} & r_{y1} & r_{z1} \\ r_{x2} & r_{y2} & r_{z2} \end{bmatrix}$$

$$\mathbf{p}' = \mathbf{R}\mathbf{p} = \begin{bmatrix} r_{x0} & r_{y0} & r_{z0} \\ r_{x1} & r_{y1} & r_{z1} \\ r_{x2} & r_{y2} & r_{z2} \end{bmatrix} \begin{pmatrix} \mathrm{p}_x \\ \mathrm{p}_y \\ \mathrm{p}_z \end{pmatrix}$$
$$= r_x \mathrm{p}_x + r_y \mathrm{p}_y + r_z \mathrm{p}_z$$

- Columns are unit vectors, and mutually orthogonal
- Columns are formed by images of coordinate axes
- Inverse == transpose, $\mathrm{R}^{-1} = \mathrm{R}^{\mathrm{T}}$
- Preserves length, angles, handed-ness

Rotation Matrix



$$\mathbf{R} = \begin{bmatrix} r_{x0} & r_{y0} & r_{z0} \\ r_{x1} & r_{y1} & r_{z1} \\ r_{x2} & r_{y2} & r_{z2} \end{bmatrix}$$

$$\mathbf{p}' = \mathbf{R}\mathbf{p} = \begin{bmatrix} r_{x0} & r_{y0} & r_{z0} \\ r_{x1} & r_{y1} & r_{z1} \\ r_{x2} & r_{y2} & r_{z2} \end{bmatrix} \begin{pmatrix} \mathrm{p}_x \\ \mathrm{p}_y \\ \mathrm{p}_z \end{pmatrix}$$
$$= r_x \mathrm{p}_x + r_y \mathrm{p}_y + r_z \mathrm{p}_z$$

- **The matrix representation is widely used for rotational motion.**
- **It's friendly for applying rotation to each vertex (by <u>matrix-vector multiplication)</u>.**
- **But it is not suitable for dynamics:**
  - **It has too much redundancy: 9 elements but only 3 DoFs.**
  - **Defining its time derivative (*rotational velocity*) is difficult.**

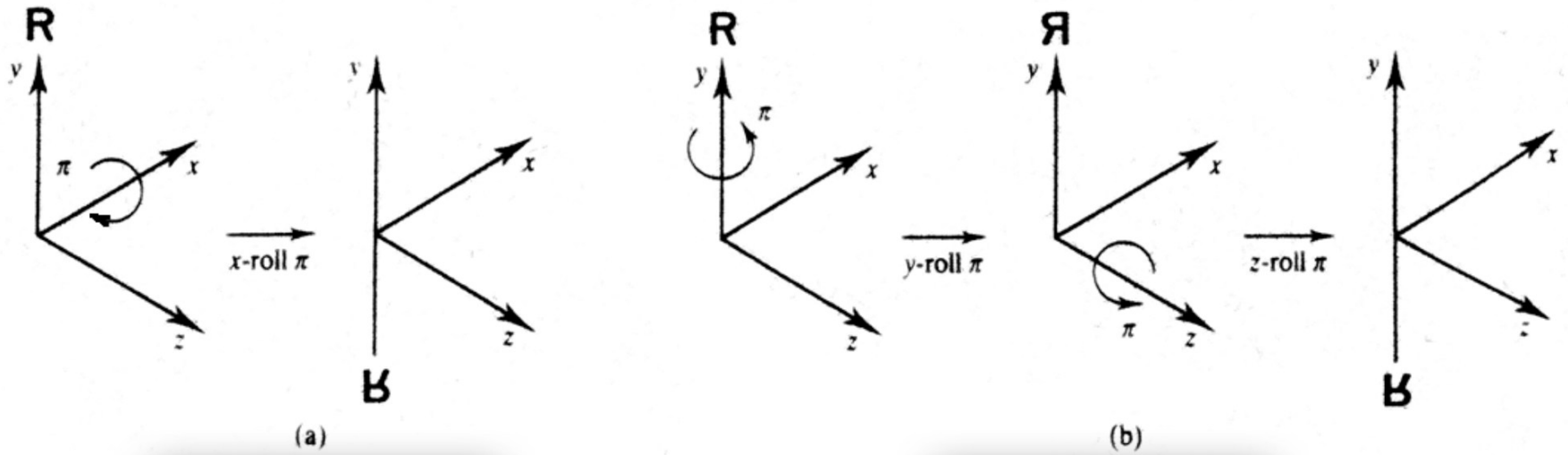
Axis and Angles

- Fixed Angles / Static Euler angles:
  - General rotation from three consecutive rotations about fixed axes
  - E.g.: x,y,z or y,z,x

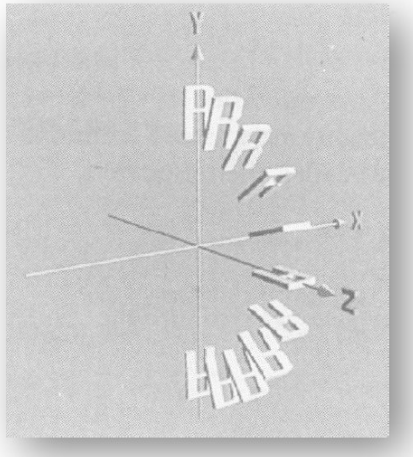
$$R = R_z R_y R_x$$

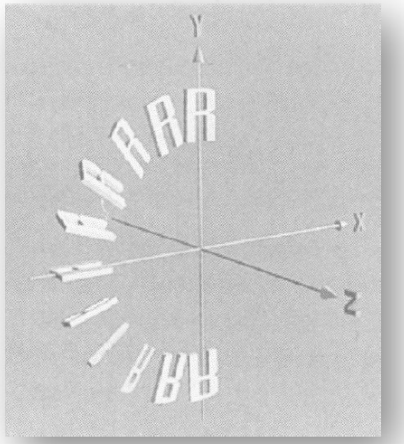
Axis and Angles - not unique...

<u>Example:</u> Fixed angles, x-y-z



Componentwise linear interpolation $(0°, 0°, 0°) \rightarrow (180°, 0°, 0°)$



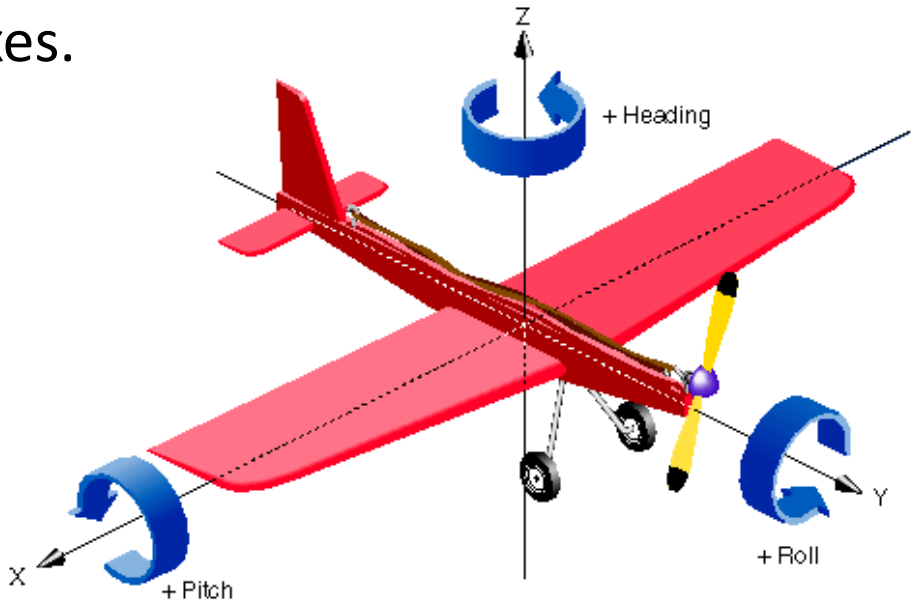


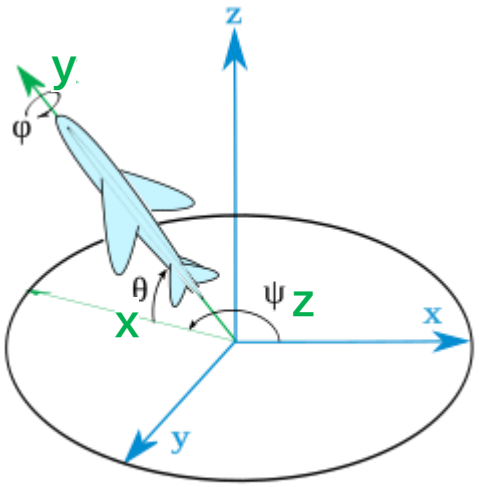
Componentwise linear interpolation $(0°, 0°, 0°) \rightarrow (0°, 180°, 180°)$

Axis and Angles

- Tait-Bryan Angles / Euler Angles:
  - Rotation from three consecutive rotations about moving frame axes.
    - ➢ Heading / Yaw = rotation z
    - ➢ Pitch = rotation x
    - ➢ Roll = rotation y

$$R = R_z R_x R_y$$

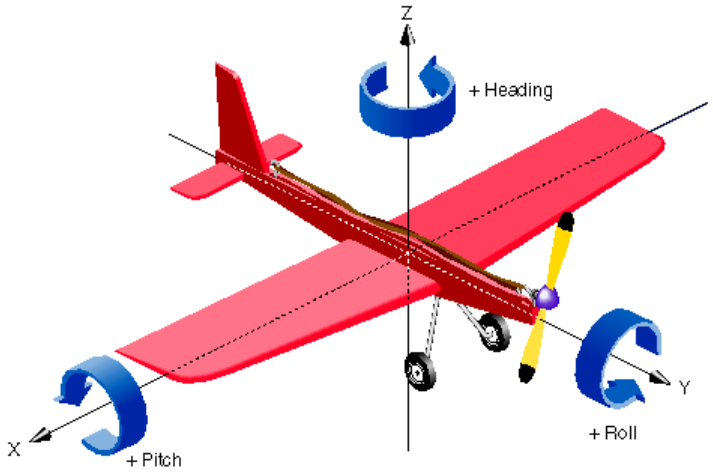


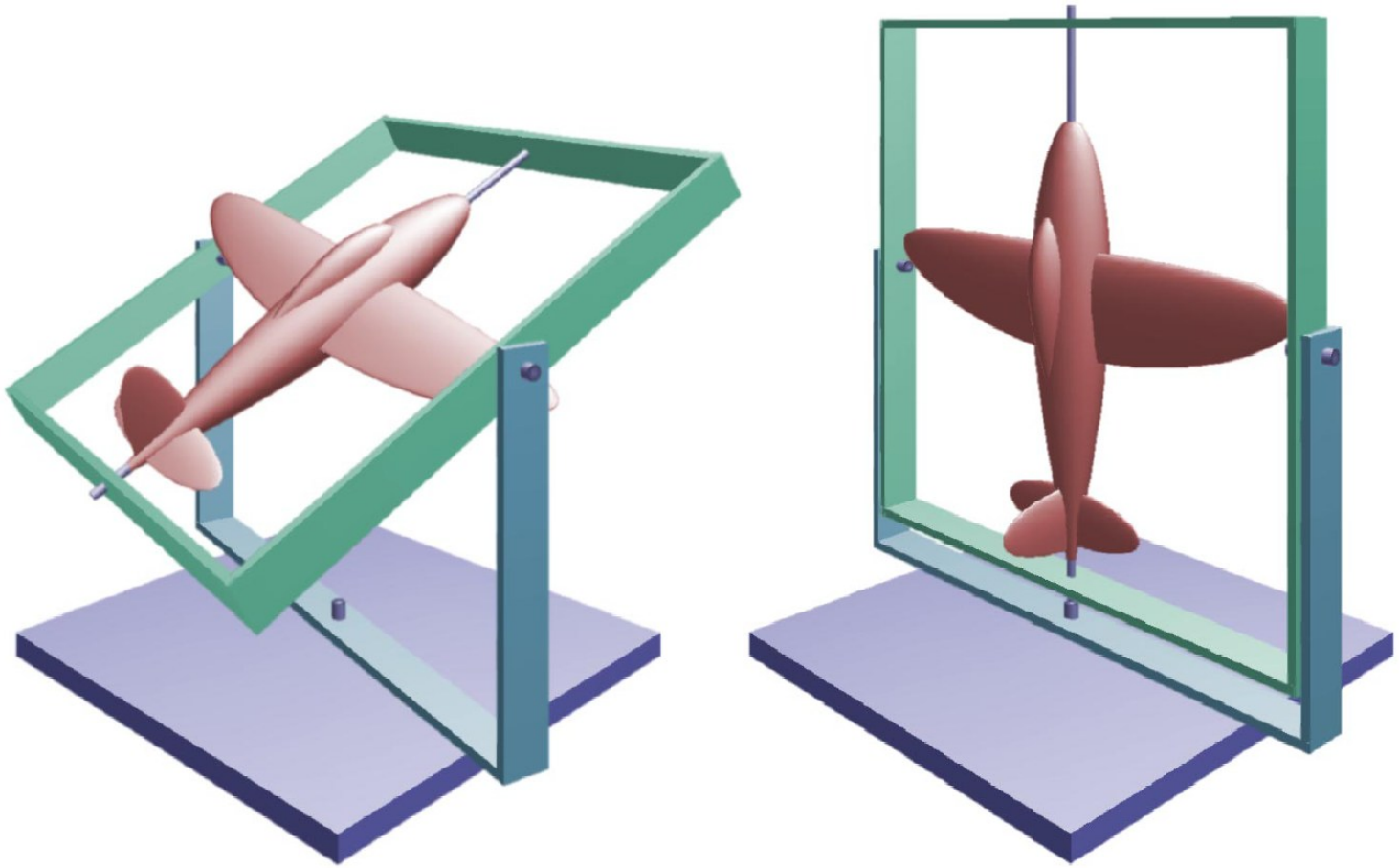


https://en.wikipedia.org/wiki/Euler_angles

Axis and Angles - Gimbal Lock

The alignment of two or more axes results in a loss of rotational DoFs.





Axis and Angles & Rotation Matrix

- Around Cartesian Axis

2D:

$$\mathrm{Rot}_\alpha = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}$$

3D:

$$R_{x,\phi} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & \text{-}\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{pmatrix}$$

$$R_{y,\phi} = \begin{pmatrix} \cos(\phi) & 0 & \sin(\phi) \\ 0 & 1 & 0 \\ \text{-}\sin(\phi) & 0 & \cos(\phi) \end{pmatrix}$$

$$R_{z,\phi} = \begin{pmatrix} \cos(\phi) & \text{-}\sin(\phi) & 0 \\ \sin(\phi) & \cos(\phi) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Axis and Angles & Rotation Matrix

- Around Arbitrary Axis

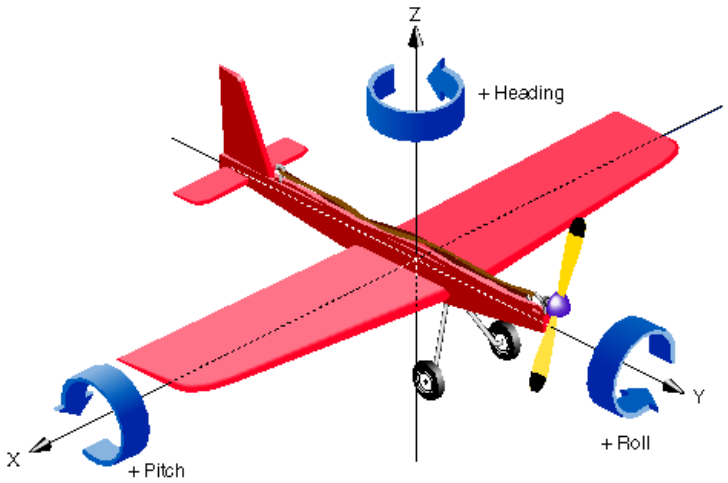
$$R_{a,\phi} = \begin{pmatrix} c + (1-c)a_x^2 & (1-c)a_x a_y - s a_z & (1-c)a_x a_z - s a_y \\ (1-c)a_x a_y - s a_z & c + (1-c)a_y^2 & (1-c)a_y a_z - s a_x \\ (1-c)a_x a_z - s a_y & (1-c)a_y a_z - s a_x & c + (1-c)a_z^2 \end{pmatrix}$$

$$\text{with } c = \cos(\phi), \text{ and } s = \sin(\phi)$$

Axis and Angles

- **Popular in design and control.**
- It is intuitive. It uses three axial rotations to represent one general rotation.

  Each axial rotation uses an angle.



- But it is not suitable for dynamics either:
  - It can lose DoFs in certain statuses: *gimbal lock*.
  - Defining its time derivative (*rotational velocity*) is difficult.
  - Numerical errors can add up (Re-orthonormalization necessary)

Quaternions

- Hamilton, Elements of Quaternions; 1866
- [Shoemake, Animating rotation with quaternion curves; SIGGRAPH 1985]
- An extension of complex number $a + bi$ (i^2=-1)

$$\mathbf{q} = (s, \mathbf{v}) = (s, x, y, z) = s1 + xi + yj + zk$$

("real" ,"imaginary")

$$s, x, y, z \in \mathbb{R}, v \in \mathbb{R}^3$$

Complex multiplications

	1	i
1	1	i
i	i	−1

Quaternion multiplications

	1	i	j	k
1	1	i	j	k
i	i	−1	k	−j
j	j	−k	−1	i
k	k	j	−i	−1

Quaternion Arithmatic

$$\mathbf{q} = (s, \mathbf{v}) = (s, x, y, z) = s1 + xi + yj + zk$$

("real" ,"imaginary") $\quad s, x, y, z \in \mathbb{R}, v \in \mathbb{R}^3$

Scalar-quaternion Multiplication $\quad a\mathbf{q} = [as \quad a\mathbf{v}]$

Addition/Subtraction $\quad \mathbf{q}_1 \pm \mathbf{q}_2 = [s_1 \pm s_2 \quad \mathbf{v}_1 \pm \mathbf{v}_2]$

Graßmann Product $\quad \mathbf{q}_1 \times \mathbf{q}_2 = (s_1 + x_1\boldsymbol{i} + y_1\boldsymbol{j} + z_1\boldsymbol{k})(s_2 + x_2\boldsymbol{i} + y_2\boldsymbol{j} + z_2\boldsymbol{k})$

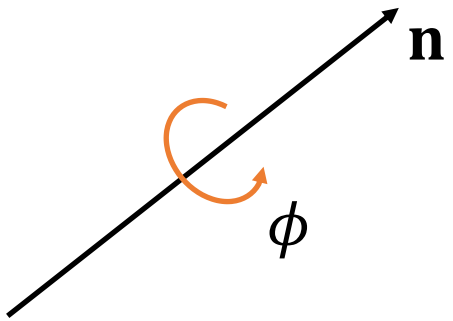
$$\mathbf{q}_1 \times \mathbf{q}_2 = [s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \quad s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2]$$

Magnitude $\quad \|\mathbf{q}\| = \sqrt{s^2 + \mathbf{v} \cdot \mathbf{v}}$

Quaternion multiplication is not commutative!

Unit-length Quaternions for Rotation

- Unit-length Quaternions represent a rotation around **n** by angle ϕ:



$$\mathbf{q} = \big(\cos(\phi/2), \mathbf{n}\,\sin(\phi/2)\big) = \begin{pmatrix} \cos(\phi/2) \\ n_x\ \sin(\phi/2)i \\ n_y\ \sin(\phi/2)j \\ n_z\ \sin(\phi/2)k \end{pmatrix}$$

$$\text{with angle } \phi,\ \text{axis } \mathbf{n} = (x, y, z)^T, |\mathbf{n}| = 1$$

Unit-length Quaternions for Rotation

- Rotate a 3D point $\mathbf{x}$ with quaternion $\mathrm{Q} = [\cos(\frac{\phi}{2}), \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)]$:
  - Conjugate quaternion Q^{-1} :

$$\mathrm{Q} = \left[\cos(\tfrac{\phi}{2}), \boldsymbol{n}\sin\left(\tfrac{\phi}{2}\right)\right]; \mathrm{Q}^{-1} = \left[\cos\left(\tfrac{\phi}{2}\right), -\boldsymbol{n}\sin\left(\tfrac{\phi}{2}\right)\right]; \ \mathrm{Q}\mathrm{Q}^{-1} = [1 \quad \mathbf{0}]$$

  - Rotated position $\mathbf{x}'$:

$$\mathrm{Q}\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\mathrm{Q}^{-1} = \begin{bmatrix}\cos(\frac{\phi}{2})\\ \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)\end{bmatrix}\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\begin{bmatrix}\cos(\frac{\phi}{2})\\ \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)\end{bmatrix}^{-1} = \begin{bmatrix}0\\ \mathbf{x}'\end{bmatrix}$$

定理：对于任意规范化四元数 $\mathrm{Q} = [\cos(\frac{\phi}{2}), \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)]$

算子 $\mathrm{L}(\mathbf{x}) = \mathrm{Q}\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\mathrm{Q}^{-1}$ 等价于使向量 $\mathbf{x}$ 绕向量 $\boldsymbol{n}$ 的正方向旋转角度ϕ

$$\mathbf{q}_1 \times \mathbf{q}_2 = [s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \quad s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2]$$

Unit-length Quaternions for Rotation

- Rotate a 3D point $\mathbf{x}$ with quaternion $\mathrm{Q} = [\cos(\frac{\phi}{2}), \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)]$:
  - Conjugate quaternion Q^{-1} :

$$\mathrm{Q} = \left[\cos(\tfrac{\phi}{2}), \boldsymbol{n}\sin\left(\tfrac{\phi}{2}\right)\right]; \mathrm{Q}^{-1} = \left[\cos\left(\tfrac{\phi}{2}\right), -\boldsymbol{n}\sin\left(\tfrac{\phi}{2}\right)\right]; \ \mathrm{Q}\mathrm{Q}^{-1} = [1 \quad \mathbf{0}]$$

  - Rotated position $\mathbf{x}'$:

$$\mathrm{Q}\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\mathrm{Q}^{-1} = \begin{bmatrix}\cos(\frac{\phi}{2})\\ \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)\end{bmatrix}\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\begin{bmatrix}\cos(\frac{\phi}{2})\\ \boldsymbol{n}\sin\left(\frac{\phi}{2}\right)\end{bmatrix}^{-1} = \begin{bmatrix}0\\ \mathbf{x}'\end{bmatrix}$$

- Consecutive rotations:
  - Two consecutive rotations $\boldsymbol{q}_1$, $\boldsymbol{q}_2$ can be expressed as a single one : $(\boldsymbol{q}_2\boldsymbol{q}_1)$

$$\boldsymbol{q}_2(\boldsymbol{q}_1\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}\boldsymbol{q}_1{}^{-1})\boldsymbol{q}_2{}^{-1} = (\boldsymbol{q}_2\boldsymbol{q}_1)\begin{bmatrix}0\\ \mathbf{x}\end{bmatrix}(\boldsymbol{q}_2\boldsymbol{q}_1)^{-1}$$

$$\mathbf{q}_1 \times \mathbf{q}_2 = [s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \quad s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2]$$

Unit-length Quaternions & Rotation Matrix

$$\mathbf{q} = \begin{bmatrix} s \\ (x, y, z) \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} s^2 + x^2 - y^2 - z^2 & 2(xy - sz) & 2(xz + sy) \\ 2(xy + sz) & s^2 - x^2 + y^2 - z^2 & 2(yz - sx) \\ 2(xz - sy) & 2(yz + sx) & s^2 - x^2 - y^2 + z^2 \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} \mathrm{R}_{11} & \mathrm{R}_{12} & \mathrm{R}_{13} \\ \mathrm{R}_{21} & \mathrm{R}_{22} & \mathrm{R}_{23} \\ \mathrm{R}_{31} & \mathrm{R}_{32} & \mathrm{R}_{33} \end{bmatrix}$$

$$s = 1/2\sqrt{1 + R_{11} + R_{22} + R_{33}}\ ,\ x = \frac{R_{32} - R_{23}}{4s}\ ,\ y = \frac{R_{13} - R_{31}}{4s}\ ,\ z = \frac{R_{21} - R_{12}}{4s}$$

Unit-length Quaternions for Rotation

- "Almost" unique representation of orientation (only q and -q describe same orientation)
- No gimbal lock
- Efficient concatenation of multiple rotations
- Its time derivative (rotational velocity) is well defined
- Disadvantages:
  - Re-normalization necessary
  - Not very intuitive...

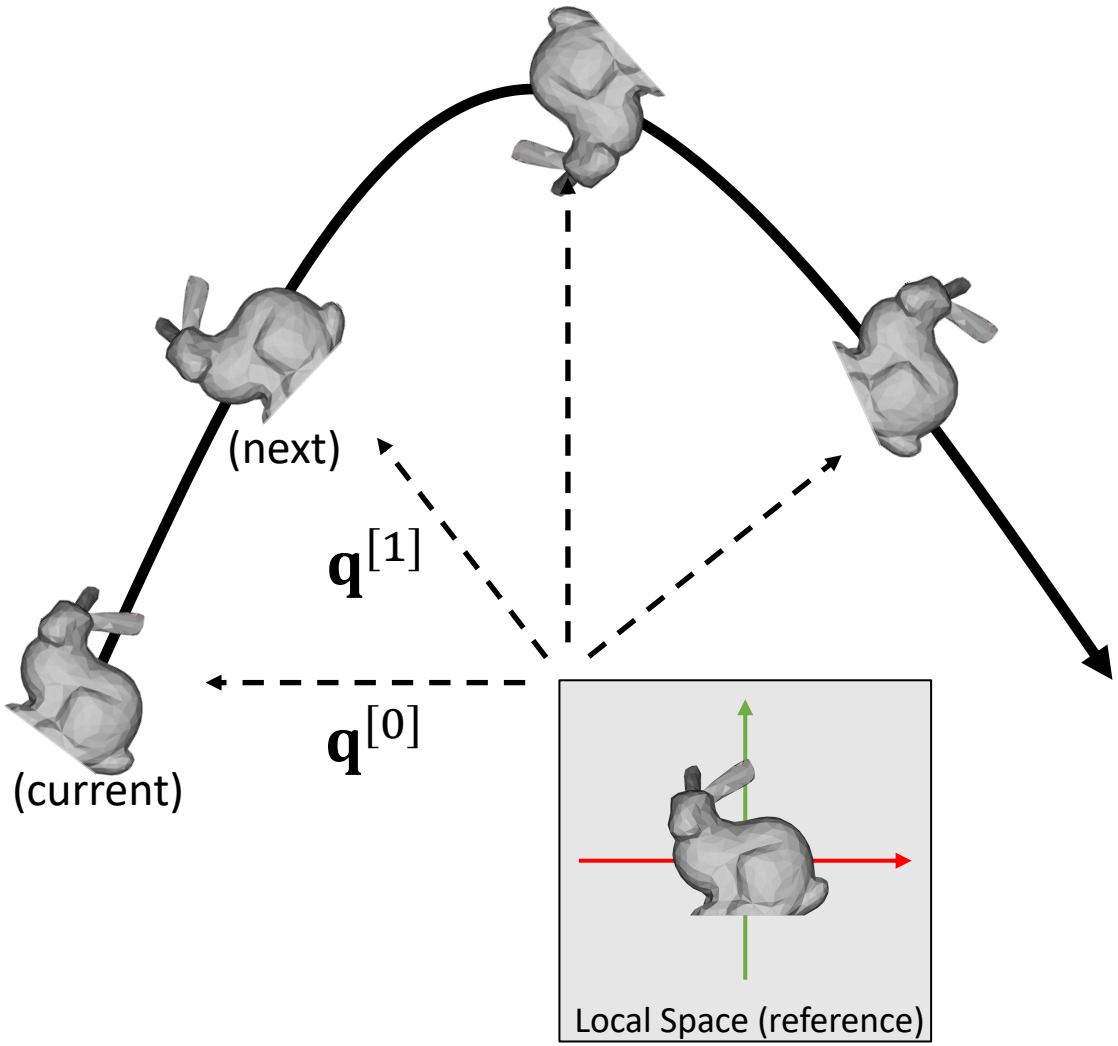
Quaternions supported in Eigen

```
// the header file
#include <Eigen/Dense>

Eigen::Quaternionf q;                // Orientation.
Eigen::Quaternionf q_copy(q.w(), q.x(), q.y(), q.z()); // Access with x(),y(),z(),w()

Eigen::AngleAxisf aa; aa = Quaternionf(..); // convert Quaternion to AngleAxis
Eigen::Matrix3f rotM = q.toRotationMatrix() ; // convert Quaternion to Rotation Matrix


rot3 = rot1.slerp(alpha,rot2); // Spherical interpolation
q.normalize();
q_copy = q.normalized();
```

Eigen: Space transformations:
https://eigen.tuxfamily.org/dox/group__TutorialGeometry.html

Rotational Motion:
2. Orientation, Time Integrate

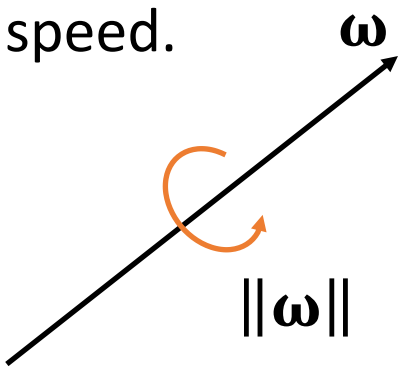
Orientation and Angular Velocity ("Kinematics")



Quaternion $\mathbf{q}$ to represent the orientation, i.e., the rotation from the *reference* to the *current*.

Angular velocity $\boldsymbol{\omega}$ tells how orientation changes:

- The direction of $\boldsymbol{\omega}$ is the axis.
- The magnitude of $\boldsymbol{\omega}$ is the speed.



Angular Momentum ("Dynamics")

- $\boldsymbol{L} = \mathbf{I}\boldsymbol{w}$ (analogy of $\mathbf{M} = m\mathbf{v}$, but $\mathbf{I}$ is a tensor in 3D)

  "$\mathbf{I}$", Inertia tensor measures how easily does the rigid body rotates

- $\frac{d}{dt}\boldsymbol{L} = \boldsymbol{\tau}$ (analogy of $\frac{d}{dt}m\mathbf{v} = \boldsymbol{f}$)

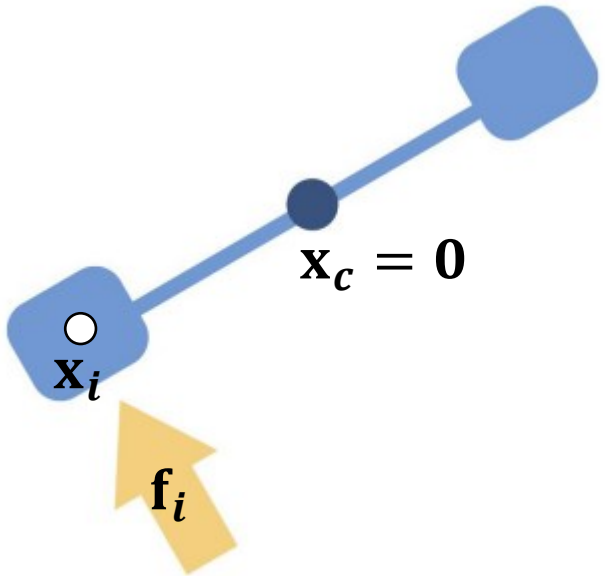
- $\boldsymbol{\tau} = \sum_i \mathbf{x}_i \times \mathbf{f}_i$

- $\mathbf{I} = \mathbf{Id}\,\mathrm{trace}(\boldsymbol{C}) - \boldsymbol{C},\ \ \boldsymbol{C} = \sum_n \mathrm{m_n}\mathbf{x}_n\mathbf{x}_n^{\mathrm{T}}$

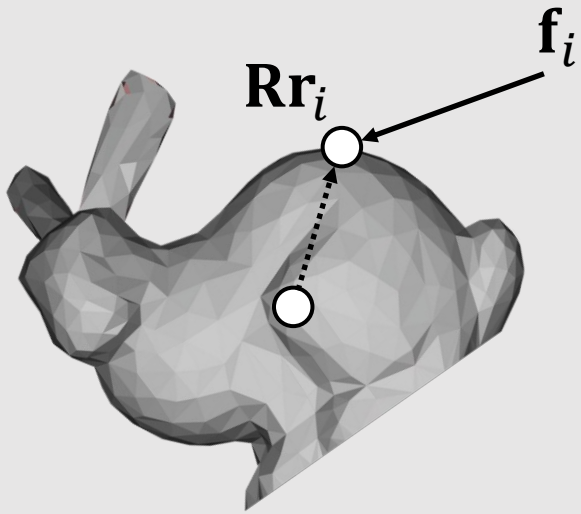
  - $\mathbf{I_t} = \mathbf{R_t}\,\mathbf{I_{ref}}\mathbf{R_t^T}$ ($\because$ $\mathbf{x}_n = \mathbf{R}_t\,\mathbf{r}_n$, $\mathbf{r}_n$ in local space)

    Inertia tensor depends on current orientation

- $\boldsymbol{\tau} \rightarrow \boldsymbol{L} \rightarrow \boldsymbol{w} \rightarrow \boldsymbol{q}$



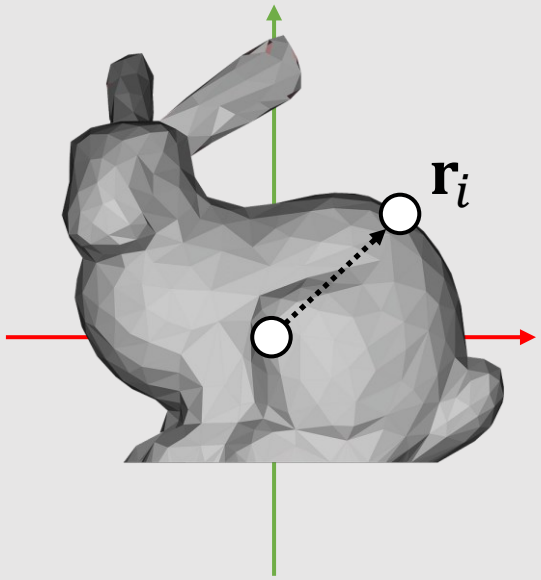
Torque and Inertia



$$\boldsymbol{\tau}_i = (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

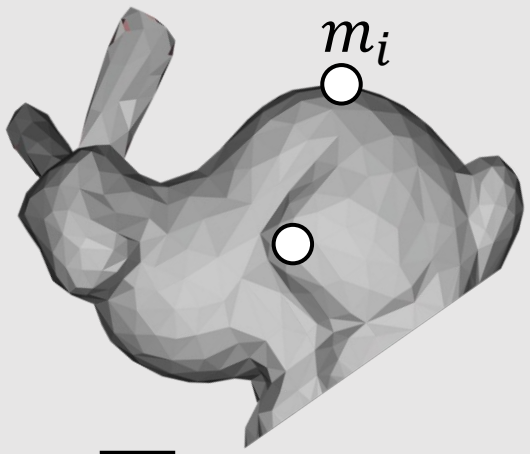
$$\boldsymbol{\tau} = \sum \boldsymbol{\tau}_i$$

The rotational equivalent of force is called torque $\boldsymbol{\tau}$.



Local Space (reference)

$$L = \sum_i m_i r_i \times (\omega \times r_i)$$



$$\mathbf{I}_{\mathbf{ref}} = \sum m_i (\mathbf{r}_i^{\mathrm{T}} \mathbf{r}_i \mathbf{Id} - \mathbf{r}_i \boldsymbol{r}_i^{\mathrm{T}})$$

$$\mathbf{I} = \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

The rotational equivalent of mass is called inertia $\mathbf{I}$.

Implementation

$\mathbf{s}^{[0]}$ → Simulator → $\mathbf{s}^{[1]}$

$\mathbf{v}_c$, $\mathbf{x}_c$

$\mathbf{f}_i \leftarrow \text{Force}(\mathbf{x}_i, \mathbf{v}_i)$

$\mathbf{f} \leftarrow \sum \mathbf{f}_i$

$\mathbf{v}_c \leftarrow \mathbf{v}_c + \Delta t M^{-1}\mathbf{f}$ ← Explicit

$\mathbf{x}_c \leftarrow \mathbf{x}_c + \Delta t \mathbf{v}$ ← Implicit

$\mathbf{v}_c$, $\mathbf{x}_c$

Translational Motion

?

Calculate Inertia tensor, torque

Update angular velocity

Updater orientation

Rotational Motion

?

Implementation

$\mathbf{s}^{[0]}$ → Simulator → $\mathbf{s}^{[1]}$

$\mathbf{v}_c$, $\mathbf{x}_c$

$$\mathbf{f}_i \leftarrow \text{Force}(\mathbf{x}_i, \mathbf{v}_i)$$

$$\mathbf{f} \leftarrow \sum \mathbf{f}_i$$

$$\mathbf{v}_c \leftarrow \mathbf{v}_c + \Delta t M^{-1}\mathbf{f} \longleftarrow \text{Explicit}$$

$$\mathbf{x}_c \leftarrow \mathbf{x}_c + \Delta t \mathbf{v} \longleftarrow \text{Implicit}$$

$\mathbf{v}_c$, $\mathbf{x}_c$

Translational Motion

$\boldsymbol{\omega}$, $\mathbf{q}$

$$\mathbf{R} \leftarrow \text{Matrix.Rotate}(\mathbf{q})$$

$$\boldsymbol{\tau}_i \leftarrow (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

$$\boldsymbol{\tau} \leftarrow \sum \boldsymbol{\tau}_i$$

$$\boldsymbol{L} \leftarrow \boldsymbol{L}_{\mathbf{old}} + \Delta t \boldsymbol{\tau}\ ;\ \mathbf{I} \leftarrow \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

$$\boldsymbol{\omega} \leftarrow (\mathbf{I})^{-1}\boldsymbol{L}$$

$$\mathbf{q} \leftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q}$$

$\boldsymbol{\omega}$, $\mathbf{q}$

Rotational Motion

Quaternion Derivative: $\frac{d\mathbf{q}}{dt} = \frac{1}{2}\begin{bmatrix} 0 \\ \mathbf{w} \end{bmatrix}\mathbf{q}$; $\mathbf{q}=(s, xi, yj, zk)$

Time derivative of orientation given as quaternion

Appendix B Quaternion Derivations

A formula for $\dot{q}(t)$ is derived as follows. Recall that the angular velocity $\omega(t)$ indicates that the body is instantaneously rotating about the $\omega(t)$ axis with magnitude $|\omega(t)|$. Suppose that a body were to rotate with a constant angular velocity $\omega(t)$. Then the rotation of the body after a period of time Δt is represented by the quaternion

$$[\cos\frac{|\omega(t)|\Delta t}{2}, \sin\frac{|\omega(t)|\Delta t}{2}\frac{\omega(t)}{|\omega(t)|}].$$

Let us compute $\dot{q}(t)$ at some particular instant of time t_0. At times $t_0 + \Delta t$ (for small Δt), the orientation of the body is (to within first order) the result of first rotating by $q(t_0)$ and then further rotating with velocity $\omega(t_0)$ for Δt time. Combining the two rotations, we get

$$q(t_0 + \Delta t) = [\cos\frac{|\omega(t_0)|\Delta t}{2}, \sin\frac{|\omega(t_0)|\Delta t}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\, q(t_0). \tag{B–1}$$

Making the substitution $t = t_0 + \Delta t$, we can express this as

$$q(t) = [\cos\frac{|\omega(t_0)|(t - t_0)}{2}, \sin\frac{|\omega(t_0)|(t - t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]q(t_0). \tag{B–2}$$

Let us differentiate $q(t)$ at time t_0. First, since $q(t_0)$ is a constant, let us differentiate

$$[\cos\frac{|\omega(t_0)|(t - t_0)}{2}, \sin\frac{|\omega(t_0)|(t - t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}].$$

At time $t = t_0$,

$$\begin{aligned}\frac{d}{dt}\cos\frac{|\omega(t_0)|(t - t_0)}{2} &= -\frac{|\omega(t_0)|}{2}\sin\frac{|\omega(t_0)|(t - t_0)}{2} \\ &= -\frac{|\omega(t_0)|}{2}\sin 0 = 0.\end{aligned} \tag{B–3}$$

Similarly,

$$\begin{aligned}\frac{d}{dt}\sin\frac{|\omega(t_0)|(t - t_0)}{2} &= \frac{|\omega(t_0)|}{2}\cos\frac{|\omega(t_0)|(t - t_0)}{2} \\ &= \frac{|\omega(t_0)|}{2}\cos 0 = \frac{|\omega(t_0)|}{2}.\end{aligned} \tag{B–4}$$

Thus, at time $t = t_0$,

$$\begin{aligned}\dot{q}(t) &= \frac{d}{dt}\left([\cos\frac{|\omega(t_0)|(t - t_0)}{2}, \sin\frac{|\omega(t_0)|(t - t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\, q(t_0)\right) \\ &= \frac{d}{dt}\left([\cos\frac{|\omega(t_0)|(t - t_0)}{2}, \sin\frac{|\omega(t_0)|(t - t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\right) q(t_0) \\ &= [0, \frac{|\omega(t_0)|}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\, q(t_0) \\ &= [0, \tfrac{1}{2}\omega(t_0)]\, q(t_0) = \tfrac{1}{2}[0, \omega(t_0)]\, q(t_0).\end{aligned} \tag{B–5}$$

The product $[0, \omega(t_0)]\, q(t_0)$ is abbreviated to the form $\omega(t_0)q(t_0)$; thus, the general expression for $\dot{q}(t)$ is

$$\dot{q}(t) = \tfrac{1}{2}\omega(t)q(t). \tag{B–6}$$

SIGGRAPH 2001 COURSE NOTES
PHYSICALLY BASED MODELING
Rigid Body Simulation
David Baraff
Pixar Animation Studios

Quaternion Derivative: $\frac{d\mathbf{q}}{dt} = \frac{1}{2}\begin{bmatrix}0\\ \mathbf{w}\end{bmatrix}\mathbf{q}$; $\mathbf{q}=(s, xi, yj, zk)$

Time derivative of orientation given as quaternion

Appendix B Quaternion Derivations

A formula for $\dot{q}(t)$ is derived as follows. Reca
is instantaneously rotating about the $\omega(t)$ axi
rotate with a constant angular velocity $\omega(t)$.
is represented by the quaternion

$$[\cos\frac{|\omega(t)|\Delta t}{2}$$

Let us compute $\dot{q}(t)$ at some particular insta
orientation of the body is (to within first orde
rotating with velocity $\omega(t_0)$ for Δt time. Con

$$q(t_0+\Delta t) = [\cos\frac{|\omega(t_0)}{2}$$

Making the substitution $t = t_0 + \Delta t$, we can e

$$q(t) = [\cos\frac{|\omega(t_0)|(t-t_0)}{2}$$

Let us differentiate $q(t)$ at time t_0. First, sinc

$$[\cos\frac{|\omega(t_0)|(t-t_0)}{2}$$

At time $t = t_0$,

$$\frac{d}{dt}\cos\frac{|\omega(t_0)|(t-t_0)}{2}$$

$$= -\frac{|\omega(t_0)|}{2}\sin 0 = 0.$$

$$s\frac{|\omega(t_0)|(t-t_0)}{2}$$
$$s\,0 = \frac{|\omega(t_0)|}{2}. \qquad \text{(B–4)}$$

$$\frac{|(t-t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\,q(t_0)\Big)$$
$$\frac{|(t-t_0)}{2}\frac{\omega(t_0)}{|\omega(t_0)|}]\Big)\,q(t_0) \qquad \text{(B–5)}$$

).

$_0)q(t_0)$; thus, the general expression for

(B–6)

$$\frac{d\mathbf{q}}{dt} = \frac{1}{2}\begin{bmatrix}0\\ \mathbf{w}\end{bmatrix}\mathbf{q}$$

$$\mathbf{q} \longleftarrow \mathbf{q} + \begin{bmatrix}0 & \frac{\Delta t}{2}\boldsymbol{\omega}\end{bmatrix} \times \mathbf{q}$$

Explicit Euler

Rigid Body Simulation
David Baraff
Pixar Animation Studios

Implementation

$\mathbf{s}^{[0]}$ → Simulator → $\mathbf{s}^{[1]}$

$\mathbf{v}_c$, $\mathbf{x}_c$

$$\mathbf{f}_i \leftarrow \text{Force}(\mathbf{x}_i, \mathbf{v}_i)$$

$$\mathbf{f} \leftarrow \sum \mathbf{f}_i$$

$$\mathbf{v}_c \leftarrow \mathbf{v}_c + \Delta t M^{-1}\mathbf{f} \quad \leftarrow \text{Explicit}$$

$$\mathbf{x}_c \leftarrow \mathbf{x}_c + \Delta t \mathbf{v} \quad \leftarrow \text{Implicit}$$

$\mathbf{v}_c$, $\mathbf{x}_c$

Translational Motion

$\boldsymbol{\omega}$, $\boldsymbol{L}$, $\mathbf{q}$

$$\mathbf{R} \leftarrow \text{Matrix.Rotate}(\mathbf{q})$$

$$\boldsymbol{\tau}_i \leftarrow (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

$$\boldsymbol{\tau} \leftarrow \sum \boldsymbol{\tau}_i$$

$$\boldsymbol{L} \leftarrow \boldsymbol{L}_{\text{old}} + \Delta t \boldsymbol{\tau}\,;\ \mathbf{I} \leftarrow \mathbf{R}\mathbf{I}_{\text{ref}}\mathbf{R}^{\mathrm{T}}$$

$$\boldsymbol{\omega} \leftarrow (\mathbf{I})^{-1}\boldsymbol{L}$$

$$\mathbf{q} \leftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q}$$

Rotational Motion

$\boldsymbol{\omega}$, $\boldsymbol{L}$, $\mathbf{q}$

Implementation

$\mathbf{s}^{[0]}$ → Simulator → $\mathbf{s}^{[1]}$

$$\tau = \frac{dL}{dt} \qquad \boldsymbol{L} = I\boldsymbol{\omega}$$

$$\frac{dL}{dt} \Rightarrow I\dot{\omega} + \omega \times I\omega$$

https://en.wikipedia.org/wiki/Euler's_equations_(rigid_body_dynamics)

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

$\boldsymbol{\omega}$, $\boldsymbol{L}$

$\mathbf{q}$

$$\mathbf{R} \longleftarrow \text{Matrix.Rotate}(\mathbf{q})$$

$$\boldsymbol{\tau}_i \longleftarrow (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

$$\boldsymbol{\tau} \longleftarrow \sum \boldsymbol{\tau}_i$$

$$\boldsymbol{L} \longleftarrow \boldsymbol{L}_{\mathbf{old}} + \Delta t\boldsymbol{\tau}\,;\ \mathbf{I} \longleftarrow \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

$$\boldsymbol{\omega} \longleftarrow (\mathbf{I})^{-1}\boldsymbol{L}$$

$$\mathbf{q} \longleftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q}$$

$\boldsymbol{\omega}$, $\boldsymbol{L}$

$\mathbf{q}$

Rotational Motion

Implementation

$\mathbf{s}^{[0]}$ → Simulator → $\mathbf{s}^{[1]}$

$$\tau = \frac{dL}{dt} \qquad \boldsymbol{L} = I\boldsymbol{\omega}$$

$$\frac{dL}{dt} \Rightarrow I\dot{\omega} + \omega \times I\omega$$

https://en.wikipedia.org/wiki/Euler's_equations_(rigid_body_dynamics)

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

1st Order ODE

$$\mathrm{A}\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y})$$

$\boldsymbol{\omega}$, $\mathbf{q}$

$$\mathbf{R} \longleftarrow \text{Matrix.Rotate}(\mathbf{q})$$

$$\boldsymbol{\tau}_i \longleftarrow (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

$$\boldsymbol{\tau} \longleftarrow \sum \boldsymbol{\tau}_i$$

$$\mathbf{I} \longleftarrow \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

$$\boldsymbol{\omega} \longleftarrow \boldsymbol{\omega} + \Delta t \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega})$$

$$\mathbf{q} \longleftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q}$$

$\boldsymbol{\omega}$, $\mathbf{q}$

Rotational Motion

Conclusion

- Rigid-Body Dynamics: Translational Motion + Rotational Motion
- Orientation (Quaternions)
- Time Integrate:

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

$\mathbf{v_c}$

$\mathbf{x_c}$

$$\mathbf{f}_i \longleftarrow \mathrm{Force}(\mathbf{x}_i, \mathbf{v}_i)$$

$$\mathbf{f} \longleftarrow \sum \mathbf{f}_i$$

$$\mathbf{v_c} \longleftarrow \mathbf{v_c} + \Delta t M^{-1}\mathbf{f} \longleftarrow \text{Explicit}$$

$$\mathbf{x_c} \longleftarrow \mathbf{x_c} + \Delta t \mathbf{v} \longleftarrow \text{Implicit}$$

Translational Motion

$\mathbf{v_c}$

$\mathbf{x_c}$

$\boldsymbol{\omega}$

$\mathbf{q}$

$$\mathbf{R} \longleftarrow \mathrm{Matrix.Rotate}(\mathbf{q})$$

$$\boldsymbol{\tau}_i \longleftarrow (\mathbf{R}\mathbf{r}_i) \times \mathbf{f}_i$$

$$\boldsymbol{\tau} \longleftarrow \sum \boldsymbol{\tau}_i$$

$$\mathbf{I} \longleftarrow \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

$$\boldsymbol{\omega} \longleftarrow \boldsymbol{\omega} + \Delta t \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega}) \longleftarrow \text{Explicit}$$

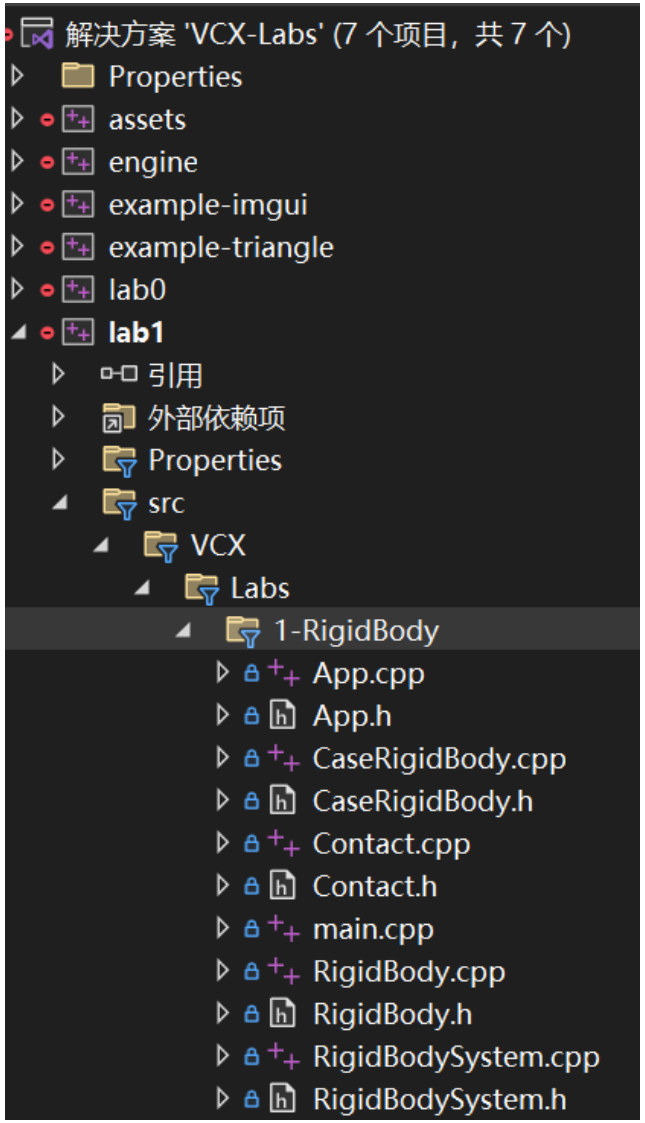
$$\mathbf{q} \longleftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q} \longleftarrow \text{Implicit}$$

Rotational Motion

$\boldsymbol{\omega}$

$\mathbf{q}$

Implementation Tips, Rigid Body System



CaseRigidBody Class (Similar to CaseBox Class) has a RigidBodySystem Member

CaseBox.cpp | CaseRigidBody.cpp | RigidBodySystem.h | CaseRigidBody.h | RigidBody.h | RigidBody.cpp | Co

{ } VCX::Labs::GettingStarted

```
}

Common::CaseRenderResult CaseBox::OnRender(std::pair<std::uint32_t, std::uint32_t> const
    // apply mouse control first
    OnProcessMouseControl(_cameraManager.getMouseMove());

    // rendering
    _frame.Resize(desiredSize);
```

CaseRigidBody.cpp | RigidBodySystem.h | CaseRigidBody.h | CaseBox.cpp | RigidBody.h | RigidBody.cpp | Contact.h

VCX::Labs::RigidBody::CaseRigidBody | OnRender(std::pair<std::uint32_t,st

```
Common::CaseRenderResult CaseRigidBody::OnRender(std::pair<std::uint32_t, std::uint32_t> const
    // apply mouse control first
    _rigidBodySystem.OnProcessMouseControl(_cameraManager.getMouseMove());

    // simulate rigid body system
    if (! _stopped) _rigidBodySystem.Advance(Engine::GetDeltaTime());

    // rendering
    _frame.Resize(desiredSize);
```

Path_of_the_code/src/VCX/Labs/1-RigidBody

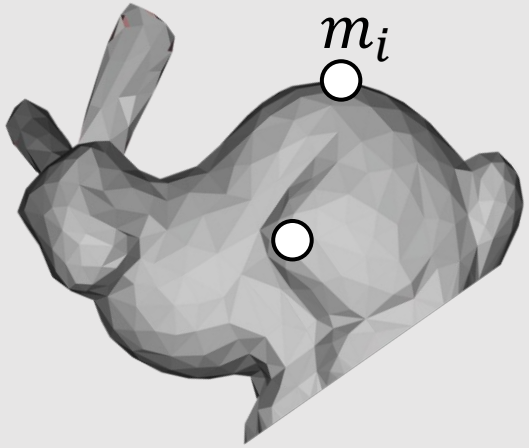
Implementation Tips

- Translational motion is much easier to implement than rotational motion.

- You can implement the update of $\mathbf{q}$ first using a constant $\boldsymbol{L}$. In that case, the object should spin constantly.

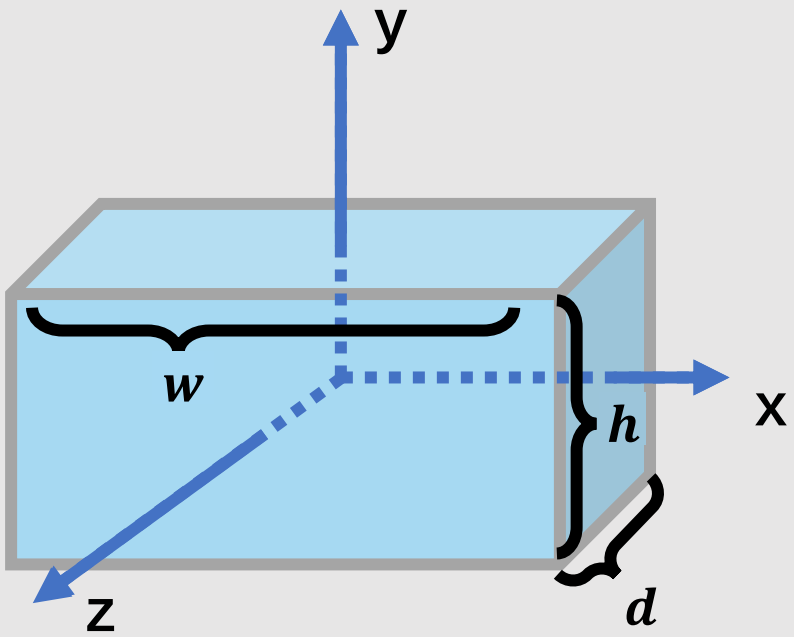
- Gravity doesn't cause any torque!

Implementation Tips



$$\mathbf{I}_{\mathbf{ref}} = \sum m_i(\mathbf{r}_i^{\mathrm{T}}\mathbf{r}_i\mathbf{Id} - \mathbf{r}_i\boldsymbol{r}_i^{\mathrm{T}})$$

$$\mathbf{I} = \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$



$$\mathbf{I}_{\mathbf{ref}} = \begin{bmatrix} \frac{1}{12}m(h^2+d^2) & 0 & 0 \\ 0 & \frac{1}{12}m(d^2+w^2) & 0 \\ 0 & 0 & \frac{1}{12}m(w^2+h^2) \end{bmatrix}$$

$$\mathbf{I} = \mathbf{R}\mathbf{I}_{\mathbf{ref}}\mathbf{R}^{\mathrm{T}}$$

Analytic formulas for inertia tensors of common objects: http://en.wikipedia.org/wiki/List_of_moment_of_inertia_tensors